

EXPERIMENT 3

Title: Global State Management using React Context API

AIM

To implement **global state management using React Context API and custom consumer Hooks** in the Carventory application, and to reduce unnecessary **prop drilling** between React components.

THEORY

In a React application, **state** represents data that can change during the execution of the application. For example, in Carventory, information such as the logged-in user, car inventory, favourite cars and comparison cars may need to be accessed by multiple components.

When data is passed from one component to another through several intermediate components, it is known as **prop drilling**. Excessive prop drilling can make an application difficult to understand and maintain.

The **React Context API** provides a mechanism for sharing data between components without passing props manually through every level of the component hierarchy.

In Carventory, different Contexts were created for different types of shared application data:

1. **AuthContext** – Manages authentication and user-related state.
2. **CarContext** – Manages shared car inventory state.
3. **FavouritesContext** – Manages the customer's favourite cars.
4. **CompareContext** – Manages cars selected for comparison.

These Contexts use **Provider components**. A Provider makes its state and related functions available to all components placed inside it.

Custom consumer Hooks are also used to make Context data easier to access. Instead of repeatedly writing Context-related code in every component, a reusable Hook can be used to access the required state.

For example, components that need favourite-car information can access the **Favourites Context** through its consumer Hook. Similarly, components requiring authentication information can use the **Auth Context**.

The **Recently Viewed** functionality continues to use the custom Hook and browser **localStorage**, as implemented in the previous experiment.

The overall provider structure allows different pages of Carventory to share important state while keeping individual components cleaner and more organized.

IMPLEMENTATION / WORKING

The Carventory application was modified to introduce global state management using React Context API.

The major Contexts implemented were:

- **AuthContext**
- **CarContext**
- **FavouritesContext**
- **CompareContext**

Provider components were placed around the application so that the required state could be accessed by different pages and components.

Custom consumer Hooks were used to provide a simple interface for accessing each Context.

For example:

- Login-related components access authentication state through **AuthContext**.
- Inventory-related components access shared car information through **CarContext**.
- Favourite-related components use **FavouritesContext**.
- Comparison-related components use **CompareContext**.

This approach reduces unnecessary prop passing and separates global state management from the presentation of the user interface.

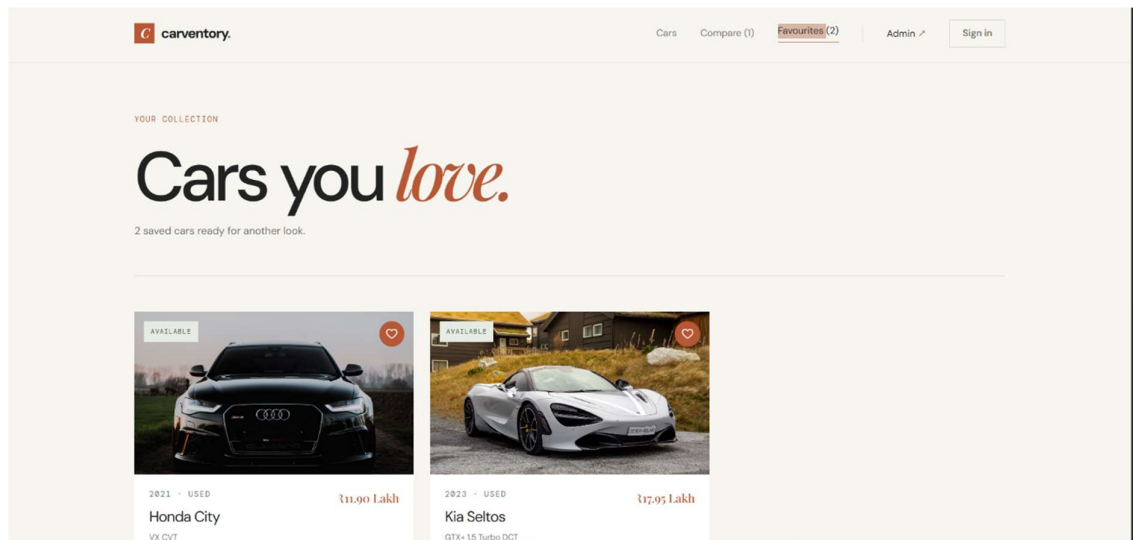
OUTPUT

The following functionality was successfully implemented:

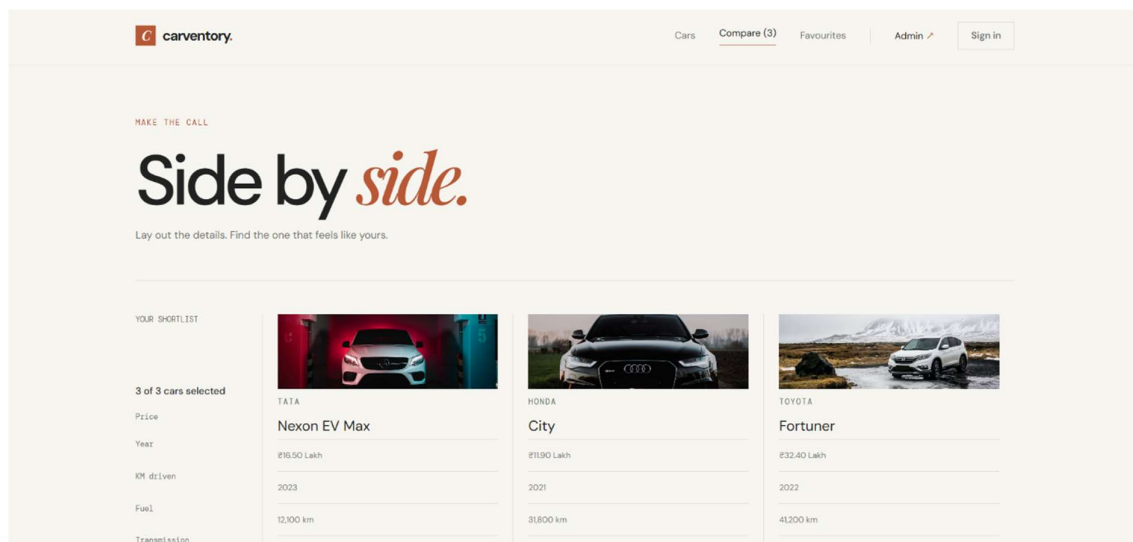
- Global authentication state
- Global car inventory state
- Global favourite-car state
- Global comparison state
- Context Providers
- Custom consumer Hooks
- Reduced prop drilling
- Recently Viewed state using Custom Hook and localStorage
- Integration with existing Carventory pages

Output Screenshot:

Favourite Feature:



COMPARISON FEATURE:



CONCLUSION

Thus, **global state management using React Context API** was successfully implemented in the Carventory application. Separate Contexts were created for authentication, car inventory, favourites and comparison.

The use of Context API and custom consumer Hooks reduced unnecessary prop drilling and made shared application state easier to manage and access. This provides a structured foundation for the backend and API integration implemented in the subsequent experiment.